

## 实验2 复杂网络基本拓扑性质分析

### 一、实验目的

- 1、学会计算节点的度、网络的平均度、度分布
- 2、学会计算网络直径、平均路径长度、网络密度等参数
- 3、学会计算节点的聚类系数，网络的平均聚类系数以及横截性

### 二、实验内容

## 实验2 复杂网络基本拓扑性质分析

### 实验2-1 节点的度、网络的平均度、度分布计算

计算无向网络节点的度、网络的平均度、度分布

```
# 创建一个无向无权网络，计算节点度、网络平均度、度分布
import networkx as nx
import matplotlib.pyplot as plt
G = nx.Graph()
G.add_nodes_from([1,2,3,4])
G.add_edges_from([(1,2),(2,3),(2,4),(3,4)])
# 获取网络G的度
d = nx.degree(G)
print("节点的度: ", d)
d = dict(nx.degree(G))
print("节点的度: ", d)
print("网络平均度为: ", sum(d.values())/len(G.nodes))
# 获取度分布
# 利用nx.degree_histogram(G)，返回所有位于区间[0, dmax]的度值的频率列表，例如返回[2,3,1]，表示度为0的节点有2个
x = list(range(max(d.values())+1))
y = [i/len(G.nodes) for i in nx.degree_histogram(G)]
# 画出度分布的图
plt.bar(x, y, width=0.1, color="blue")
plt.xlabel("$k$")
plt.ylabel("$P(k)$")
plt.xlim([0,4])
plt.show()
```

```
: # 创建一个无向加权网络，计算节点的加权度、边权、网络的平均度
import networkx as nx
import matplotlib.pyplot as plt
WG = nx.Graph()
WG.add_nodes_from([1,2,3,4,5,6])
WG.add_weighted_edges_from([(1,2,3.0),(1,3,1.0),(2,4,4.0),(2,5,1.5),(3,5,2.0),(3,6,3.5),(4,5,2.5),(4,6,0.5)])
w = [WG[e[0]][e[1]]['weight'] for e in WG.edges()]
nx.draw(WG, node_size=500, width=w, with_labels=True)
# 获取网络节点的加权度（即点权）
print("网络节点的加权度")
print(nx.degree(WG, weight='weight'), end='\n')
# 获取每条边的权重（即边权）
print("网络的边权")
for e in WG.edges():
    print(WG[e[0]][e[1]]['weight'], end=' ')
# 获取平均度
d = dict(nx.degree(WG, weight='weight'))
print("\n网络平均度为: ", sum(d.values())/len(WG.nodes))
plt.show()
```

## 计算有向网络节点的入度、出度、网络的平均度

```
#有向加权网络的节点入度和出度、网络的平均度
import networkx as nx
import matplotlib.pyplot as plt
DG=nx.DiGraph()
DG.add_nodes_from([1,2,3,4,5,6])
DG.add_weighted_edges_from([(1,2,2),(1,3,5),(2,4,2),(3,5,1),(4,6,3),(5,2,8),(5,4,2),(5,3,1),(3,6,2)])
w=[DG[e[0]][e[1]]['weight'] for e in DG.edges()]
print("节点的入度",DG.in_degree(DG, weight='weight'),end='\n')
print("节点的出度",DG.out_degree(DG, weight='weight'),end='\n')
# 获取平均入度
d1 = dict(DG.in_degree(DG, weight='weight'))
print("网络的平均入度为: ", sum(d1.values())/len(DG.nodes))
# 获取平均出度
d2 = dict(DG.out_degree(DG, weight='weight'))
print("网络的平均出度为: ", sum(d2.values())/len(DG.nodes))

nx.draw(DG,node_color="green", edge_color="red",width=w,node_size=500,with_labels=True)
plt.show()
```

## 计算真实网络的节点的度、网络的平均度、度分布

```
#真实网络度分布直方图
import numpy as np
import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
import csv

plt.rc('font', family='SimHei') # 绘图时显示中文字体,在绘制图形时,如果涉及中文文本显示,
#很多情况下默认字体可能无法正确显示中文,会出现乱码。plt.rc函数可以确保图形中的中文文本正常显示,避免乱码
df = pd.read_csv("d:\\actor_edges.csv")
G = nx.from_pandas_edgelist(df, 'from', 'to', 'weight', create_using = nx.Graph())

# 获取网络节点的加权重
print("网络节点的加权重")
print(nx.degree(G, weight='weight'),end='\n')
# 获取每条边的权重（即边权）

# 获取平均度
d = dict(nx.degree(G, weight='weight'))
print("\n网络平均度为: ", sum(d.values())/len(G.nodes))
```

和下一页的第一个图片同属一个程序

```

#定义函数get_pdf计算度分布
def get_pdf(G):
    all_k = [G.degree(i,weight='weight') for i in G.nodes()]#使用列表推导式遍历图 G 中的所有节点 i
    k = list(set(all_k)) # set函数将all_k转换为集合以去除重复的度值, 并获取所有可能的度值
    N = len(G.nodes())

    Pk = []
    for ki in sorted(k):
        c = 0
        for i in G.nodes():
            if G.degree(i,weight='weight') == ki:
                c += 1
        Pk.append(c/N)

    return sorted(k), Pk

mid_d, p = get_pdf(G)

# 绘制网络G的度分布直方图
width=0.5 # 根据情况做适当调整
plt.figure(figsize=(8,6))
plt.bar(mid_d, p, width, color='b')
plt.title("度分布直方图",fontsize=18)
plt.xlabel("k",fontsize=18)
plt.ylabel("P(k)",fontsize=18)
plt.tick_params(labelsize=18) # 设置刻度标签字体大小
plt.savefig("fig.png", dpi=600) # dpi为图片的分辨率
plt.show()

```

## 实验2-2 网络直径、最短路径、最短路径长度、网络密度、聚类系数等拓扑参数的计算

```

#计算网络的直径,最短路径, 最短路径长度, 效率, 平均路径长度, 网络密度
import networkx as nx
import matplotlib.pyplot as plt
G = nx.Graph()
G.add_nodes_from([1,2,3,4,5])
G.add_edges_from([(1,2),(2,3),(2,5),(3,4),(4,5)])
print("网络的直径为:",nx.diameter(G))
print("节点1和4的最短路径",nx.shortest_path(G, source=1, target=4))
print("节点1和4的所有最短路径",list(nx.all_shortest_paths(G, source=1, target=4)))
print("节点1和4的最短路径长度",nx.shortest_path_length(G, source=1, target=4))
print("网络的平均路径长度",nx.average_shortest_path_length(G))
print("网络的密度为:",nx.density(G))
nx.draw(G, node_size=500, with_labels=True)
plt.show()

```

: #非连通图的最短路径长度计算

```

import networkx as nx

# 创建一个非连通图
G = nx.Graph()
G.add_edges_from([(1, 2), (2, 3)])
G.add_node(4)

# 获取所有连通分量
connected_components = list(nx.connected_components(G))

# 计算每个连通分量的平均最短路径长度
for i, component in enumerate(connected_components):
    subgraph = G.subgraph(component)
    avg_shortest_path = nx.average_shortest_path_length(subgraph)
    print(f"第 {i + 1} 个连通分量的平均最短路径长度: {avg_shortest_path}")

```

```

#计算节点的聚类系数，网络的聚类系数，网络的横截性
import networkx as nx
import matplotlib.pyplot as plt
G = nx.Graph()
G.add_nodes_from([1,2,3,4,5])
G.add_edges_from([(1,2),(1,3),(1,4),(1,5),(2,3),(2,4),(2,5),(3,4),(3,5),(4,5)])
plt.figure(figsize=(8,3))
plt.subplot(131)
nx.draw(G, node_size=500, with_labels=True)
print("CC of node 1: ",nx.clustering(G, 1))

G.remove_edges_from([(2,4),(3,4),(3,5)])
# 平均聚类系数
print("平均聚类系数",nx.average_clustering(G))
# 全局聚类系数
print("全局聚类系数",nx.transitivity(G))

plt.subplot(132)
nx.draw(G, node_size=500, with_labels=True)
print("CC of node 1: ",nx.clustering(G, 1))

G.remove_edges_from([(2,3),(2,5),(4,5)])
plt.subplot(133)
nx.draw(G, node_size=500, with_labels=True)
print("CC of node 1: ",nx.clustering(G, 1))

```

```

#真实网络的直径, 平均路径长度, 网络密度, 节点的聚类系数, 网络的聚类系数, 网络的横截性
import numpy as np
import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
import csv
df = pd.read_csv("d:\\actor_edges.csv")
G = nx.from_pandas_edgelist(df, 'from', 'to', 'weight', create_using = nx.Graph())

print("网络的直径为: ",nx.diameter(G))
print("网络的平均路径长度",nx.average_shortest_path_length(G))
print("网络的密度为: ",nx.density(G))
print("节点的聚类系数: ",nx.clustering(G))
print("网络的平均聚类系数",nx.average_clustering(G))
print("网络的全局聚类系数",nx.transitivity(G))

```

### 三、实验报告内容



实验2-学号-姓名.d  
ocx